

FOR BACKEND ENGINEERS

Don't Shard It Yet.

Database scaling, measured: index, replica, cache, partition, shard

Every scaling technique moves the bottleneck and sends a bill. Find the bottleneck first.

Ten million orders, one slow query, and each common scaling technique measured in turn on one machine, starting from the question that comes before all of them: where is my bottleneck?

Every number below was measured, not assumed, and one script in the repository rebuilds the lot from an empty machine.

Versions and defaults change. Everything on this sheet was true on 28 September 2026: PostgreSQL 17.11, Redis 7.4.11, Spring Boot 4.1.1, Java 25. The method is the durable part: measure where the time goes, then choose the bill.

In scope

- Postgres 17 query plans, composite index column order and the write cost of indexes
- A streaming replica, replica lag, and read-your-writes routing in Spring
- Cache aside with Redis, invalidation, and the stampede when a hot key expires
- Monthly range partitions, pruning, and DROP against DELETE
- What resharding does to key ownership: modulo against consistent hashing

Not measured here

- Write throughput across real shards
- Cross-shard queries, transactions and global uniqueness
- Production hardware, networks and failover

One laptop, Docker Compose, synthetic data with a fixed seed.

01 The findings, in one table

NO INDEX, 10M rows	214.49 ms, parallel sequential scan, one row returned
INDEX (customer, status, created_at)	0.06 ms, 6 pages
SAME COLUMNS, created_at first	220.73 ms, 386 MB index, not used
100,000 INSERTS: 0 / 1 / 3 indexes	107.5 / 358.4 / 829.5 ms
REPLICA LAG, 2M row update	824 ms worst sample; 0 idle
STAMPEDE, 500 callers, 10 runs	500 loads, 2 with local single flight, 1 through Redis
PARTITION KEY in the query / not	0.07 / 0.25 ms
ONE MONTH, 822,079 rows	DELETE 305.158 ms, DROP partition 1.126 ms
4 SHARDS TO 5, 1M keys	modulo 80.02% moved; ring 23.55% (shards 5.0 to 27.7%); 200 vnodes 19.93%

Medians of warmed runs unless stated.

02 Measure, then index the question you ask

EXPLAIN (ANALYZE, BUFFERS) showed three workers throwing away 3.3 million rows each to return one. Slow is not the diagnosis; the plan is. The composite index works because its leading columns match the equality filters in this query, and the planner can read the newest rows first.

● ● ● The index that matches the question

```
CREATE INDEX orders_idx ON orders (customer_id, status, created_at);  
-- The same three columns, created_at first: 386 MB, and the planner ignores it  
CREATE INDEX orders_idx ON orders (created_at, status, customer_id);
```

Every secondary index needs an entry per inserted row: 100,000 inserts took 107.5 ms with none, 358.4 ms with one and 829.5 ms with three.

Measured on 2026-09-28. Reproduce it with `scripts/verify.sh` in the repository.

03 Read replicas, and the reads that must not use them

Streaming replication is asynchronous by default. A replica adds a server for eligible reads; in this setup it adds no write capacity. With replay paused on purpose, the primary said PAID and the replica said PENDING. The fix is routing: a request that must read its own write goes to the primary.

ReadRouting.java

```
public enum Target { PRIMARY, REPLICA }

/** Set for the current request when the caller must see its own writes. */
public static final ThreadLocal<Boolean> READ_YOUR_WRITES =
    ThreadLocal.withInitial(() -> false);

static class Router extends AbstractRoutingDataSource {
    @Override
    protected Object determineCurrentLookupKey() {
        boolean readOnly =
            TransactionSynchronizationManager.isCurrentTransactionReadOnly();
        return readOnly && !READ_YOUR_WRITES.get()
            ? Target.REPLICA
            : Target.PRIMARY;
    }
}

@Bean
@Primary
DataSource dataSource(@Value("${primary.url}") String primary,
    @Value("${replica.url}") String replica,
    @Value("${db.username}") String user,
    @Value("${db.password}") String password) {
    var primaryDb = pool(primary, user, password);
    var router = new Router();
    router.setTargetDataSources(Map.of(
        Target.PRIMARY, primaryDb,
        Target.REPLICA, pool(replica, user, password)));
    router.setDefaultTargetDataSource(primaryDb);
    router.afterPropertiesSet();
    // The connection is only fetched once the transaction's read-only flag is
    // known.
    return new LazyConnectionDataSourceProxy(router);
}
```

Same paused replica, same write: the read-your-writes request was served by the primary (PAID), the plain read by the replica (PENDING).

Measured on 2026-09-28. Reproduce it with scripts/verify.sh in the repository.



04 Cache aside, invalidation and the stampede

Write Postgres, then delete the cached copy, so the next read reloads. When a hot key expires, every caller misses at once. Coalescing inside a process gives one load per instance; a Redis lock with an owner token gives one load across instances, provided the winner re-checks the cache after taking the lock and releases only its own lock.

ProductCache.java: write, then delete the cached copy

```
public void changePrice(long id, int priceCents) {
    db.sql("UPDATE products SET price_cents = :p WHERE id = :id")
        .param("p", priceCents)
        .param("id", id)
        .update();
    redis.delete("product:" + id);
}
```

ProductCache.java: cross-instance single flight

```
/** Delete the lock only if we still own it, in one atomic step on the server. */
private static final RedisScript<Long> RELEASE = RedisScript.of("""
    if redis.call('get', KEYS[1]) == ARGV[1] then
        return redis.call('del', KEYS[1])
    end
    return 0""", Long.class);

private String singleFlightAcrossInstances(String key, long id) {
    String lock = "lock:" + key;
    String token = UUID.randomUUID().toString();
    if (Boolean.TRUE.equals(
        redis.opsForValue().setIfAbsent(lock, token, Duration.ofSeconds(5)))) {
        try {
            // Re-check: the previous winner may have filled the cache and
            // released the lock between our miss and our lock.
            String filled = redis.opsForValue().get(key);
            return filled != null ? filled : loadAndCache(key, id);
        } finally {
            redis.execute(RELEASE, List.of(lock), token);
        }
    }
    // Lost the race: wait for the winner to fill the cache rather than load.
    for (int i = 0; i < 250; i++) {
        String cached = redis.opsForValue().get(key);
        if (cached != null) return cached;
        sleep(20);
    }
    return loadAndCache(key, id);
}
```



500 callers across two instances, one expired key, ten runs: 500 loads without coordination, 2 with in-process single flight, 1 through Redis, every run.

Measured on 2026-09-28. Reproduce it with `scripts/verify.sh` in the repository.

05 Partitions: less data considered, and cheap retention

Partitioning is not a speed button. With the partition key in the query, Postgres prunes to one partition; without it, it scans an index in each of twelve. Both were fast here. Where it earns its place is data that splits by time.

sql/05-partition.sql

```
DROP TABLE IF EXISTS orders_p;
CREATE TABLE orders_p (LIKE orders) PARTITION BY RANGE (created_at);
DO $$
BEGIN
  FOR m IN 1..12 LOOP
    EXECUTE format(
      'CREATE TABLE orders_p_2026_%s PARTITION OF orders_p '
      || 'FOR VALUES FROM (%L) TO (%L)',
      lpad(m::text, 2, '0'),
      make_timestampz(2026, m, 1, 0, 0, 0, 'UTC'),
      CASE WHEN m = 12
        THEN make_timestampz(2027, 1, 1, 0, 0, 0, 'UTC')
        ELSE make_timestampz(2026, m + 1, 1, 0, 0, 0, 'UTC')
      END);
  END LOOP;
END $$;
INSERT INTO orders_p SELECT * FROM orders;
CREATE INDEX ON orders_p (customer_id);
```

Removing one month, 822,079 rows: DELETE 305.158 ms, DROP of the partition 1.126 ms.

Measured on 2026-09-28. Reproduce it with `scripts/verify.sh` in the repository.

06 Sharding: the bill for changing the number of shards

Modulo routing reassigned 80% of keys when four shards became five. A ring with one point per shard moved 23.55% but left the shards badly unbalanced. Two hundred virtual points per shard moved 19.93% and kept every shard close to a fifth. This is key ownership only; moving the data, cross-shard queries and transactions are further bills not measured here.

sim/Resharding.java: the two routers

```
static Router modulo(int shards) {
    return h -> (int) Long remainderUnsigned(h, shards);
}

static Router ring(int shards, int vnodes) {
    TreeMap<Long, Integer> ring = new TreeMap<>();
    for (int s = 0; s < shards; s++)
        for (int v = 0; v < vnodes; v++)
            ring.put(hash("shard-" + s + "#" + v), s);
    return h -> {
        Map.Entry<Long, Integer> e = ring.ceilingEntry(h);
        return (e != null ? e : ring.firstEntry()).getValue();
    };
}
```

1,000,000 keys, 4 to 5 shards. Ideal for a balanced consistent hash: 20% moved.

sim/Resharding.java in the repository: `java sim/Resharding.java`

07 The questions this answers

Why does composite index column order matter, and how do you tell from a plan?

What does a read replica not give you?

How do you give a user read-your-writes with an asynchronous replica?

What is a cache stampede, and why must a lock winner re-check the cache?

Why does modulo sharding make resharding expensive, and what do virtual nodes fix?

Before you scale out

1. Read the plan with EXPLAIN (ANALYZE, BUFFERS) before choosing a fix
2. Keep the indexes your queries use, and question the ones they do not
3. Route reads that must see their own writes to the primary
4. Invalidate on write, and coordinate misses on a hot key
5. Partition data that splits by time, and query with the partition key
6. Shard only when one database cannot carry the writes or the data

Everything else

 Site https://code-with-sam-dev.github.io	 Facebook https://www.facebook.com/codewithsamdev
 YouTube https://www.youtube.com/@CodewithSam-Dev	 GitHub https://github.com/code-with-sam-dev
 LinkedIn https://www.linkedin.com/company/code-with-sam-dev	 Buy me a coffee https://buymeacoffee.com/codewithsam
 x https://x.com/CodeWithSamDev	 Buy me sadza https://buymesadza.com/codewithsam
 TikTok https://www.tiktok.com/@codewithsamdev	Every command, plan and number https://github.com/code-with-sam-dev/database-scaling-measured
 Instagram https://www.instagram.com/codewithsamdev	PostgreSQL: using EXPLAIN https://www.postgresql.org/docs/17/using-explain.html

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

PostgreSQL is a trademark of the PostgreSQL Community Association. Redis is a trademark of Redis Ltd. Spring is a trademark of Broadcom. Java is a trademark of Oracle. This is an independent, unofficial build produced by Code with Sam, not affiliated with or endorsed by any of them.

Technical claims verified 2026-09-28 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.